

Zenei Jelfeldolgozás Házi Feladat Dokumentáció

Apró Balázs(D7K0CV)

Feladateleírás:

C/M-6 Nemlineáris gitáreffekt megvalósítása Paraméterezhető statikus nemlinearitás (torzító) modellezése. Az átlapolódó komponensek csökkentése céljából az effekt fusson magasabb mintavételi frekvencián, azaz a bememőjelet interpolálja, majd feldolgozás után decimálja. Az interpoláló és decimáló szűrőket MATLAB segítségével tervezze. Ellenőrző pont: statikus karakterisztika megvalósítása túlmintavételezés nélkül.

1. Bevezetés és Működési Elv

A **NonlinearDistortionAudioProcessor** egy VST/AU formátumú gitár/audio effekt plugin, amely a bejövő jelen **nemlineáris torzítást (Hard Clipping)** végez.

A digitális torzítások legnagyobb problémája az **aliasing** (spektrális átlapolódás). Mivel a torzítás (pl. négyszögjelesítés) végtelen sok felharmonikust generál, ezek könnyen túlléphetik a Nyquist-frekvenciát (a mintavételi frekvencia felét), és "visszatükröződnek" a hallható tartományba, kellemetlen, digitális zajt okozva.

Ennek kiküszöbölésére a plugin **túlmintavételezést (Oversampling)** alkalmaz:

1. A jelet felskálázza (Upsampling) $L=4$ \$arányban.
2. Ezen a magasabb frekvencián végzi el a torzítást (így a keletkező harmonikusok nem érik el az új, magasabb Nyquist határt).
3. Kiszűri a magas frekvenciákat (Anti-aliasing filter).
4. Visszaskálázza a jelet az eredeti frekvenciára (Downsampling).

2. Inicializálás és Szűrőtervezés (prepareToPlay)

A plugin elindulásakor (illetve a mintavételi frekvencia változásakor) a prepareToPlay metódus fut le. Itt történik a FIR (Véges Impulzusválaszú) szűrők beállítása és a memórafoglalás.

2.1. Szűrő Együtthatók (Coefficients)

A kódban egy előre kiszámolt, aluláteresztő FIR szűrő együtthatóit használjuk (firCoeffs). Ez a szűrő felelős azért, hogy levágja a magas frekvenciákat az interpoláció és a decimálás során.

2.2. Gain Staging (Jelszint kompenzáció)

Ez a kód egyik legkritikusabb része. A túlmintavételezés során "zero stuffing" módszert alkalmazunk (nullákat szúrunk a minták közé). Ez a jel energiáját az $1/L$ részére csökkenti.

- **Upsampling szűrő:** A kimeneti jelszint helyreállításához a szűrő együtthatóit meg kell szorozni L -lel (jelen esetben 4-gyel).
- **Downsampling szűrő:** Itt nem szükséges erősítés, az együtthatók összegének 1.0-nak (Unity Gain) kell maradnia.

Kódrészlet:

```
void NonlinearDistortionAudioProcessor::prepareToPlay(double sampleRate, int samplesPerBlock)
{
    int L = 4;

    // --- 1. KOEFFICIENSEK BETÖLTÉSE ---
    // Helyi vektorba másoljuk az adatokat
    std::vector<float> coeffsUpsample(std::begin(firCoeffs), std::end(firCoeffs));
    std::vector<float> coeffsDownsample(std::begin(firCoeffs), std::end(firCoeffs));

    // Upsample szűrő: Gain kompenzáció (szorzás L-lel)
    // Mivel a "zero stuffing" leosztja az energiát, a szűrőnek kell visszahoznia.
    for (auto& c : coeffsUpsample)
        c *= (float)L;

    // Downsampling szűrő: Hagyjuk Unity Gain-en (összeg ~1.0), nem kell módosítani.
    // (A te kódban osztva volt, ami halkulást okozott volna)

    // JUCE specifikus pointer létrehozása
    auto upState = new juce::dsp::FIR::Coefficients<float>(coeffsUpsample.data(), coeffsUpsample.size());
    auto downState = new juce::dsp::FIR::Coefficients<float>(coeffsDownsample.data(), coeffsDownsample.size());

    upsampleFIR.state = upState;
    downsampleFIR.state = downState;
}
```

Matlab-ból kimásoltoefficiensek:

```
float firCoeffs[] = {
    -0.0001f, 0.0004f, 0.0014f, -0.0034f, -0.0073f,
    0.0139f, 0.0247f, -0.0423f, -0.0729f, 0.1392f,
    0.4464f, 0.4464f, 0.1392f, -0.0729f, -0.0423f,
    0.0247f, 0.0139f, -0.0073f, -0.0034f, 0.0014f,
    0.0004f, -0.0001f
};
```

3. A Jelfeldolgozási Lánc (processBlock)

A processBlock minden audio blokk (buffer) esetén lefut. Itt történik a tényleges manipuláció. A folyamat 5 fő lépésre bontható.

3.1. Interpoláció (Zero Stuffing)

Az eredeti jel mintái közé $L-1$ darab nullát illesztünk. Ezzel a mintavételi frekvencia a négyszeresére nő, de a spektrumban megjelennek az eredeti jel "tükörképei" (imaging).

Kódrészlet (processBlock és segédfüggvény):

```
// --- 3. BUFFER ALLOKÁCIÓ ---
// Itt foglalunk memóriát, nem a processBlock-ban!
upsampledBuffer.setSize(getTotalNumOutputChannels(), samplesPerBlock * L);
downsampledBuffer.setSize(getTotalNumOutputChannels(), samplesPerBlock); // Opcionális, ha csak helyben használnád
```

Itt lefoglalunk egy kellő nagyságú buffert az interpolált elemeknek és a később decimáltaknak is.

```

void NonlinearDistortionAudioProcessor::fillUpsampleBuffer(
    const juce::AudioBuffer<float>& input,
    juce::AudioBuffer<float>& upsampled,
    int L)
{
    int numInputSamples = input.getNumSamples();
    int numChannels = input.getNumChannels();

    upsampled.clear();

    for (int ch = 0; ch < numChannels; ++ch)
    {
        auto* in = input.getReadPointer(ch);
        auto* out = upsampled.getWritePointer(ch);

        for (int n = 0; n < numInputSamples; ++n)
            out[n * L] = in[n];
    }
}

```

Ennek a függvénynek a segítségével másoljuk az eredeti bemeneti buffer mintáit az túlmintavételezett bufferbe.

```

if (upsampledBuffer.getNumSamples() < numInputSamples * L)
    upsampledBuffer.setSize(buffer.getNumChannels(), numInputSamples * L);

upsampledBuffer.clear();
// downsampledBuffer-t nem feltétlen kell clear-elni, mert felülírjuk, de biztonságosabb:
// downsampledBuffer.setSize(buffer.getNumChannels(), numInputSamples); // Ha kellene

// 1. Interpoláció (Zero stuffing)
fillUpsampleBuffer(buffer, upsampledBuffer, L);

```

Itt leellenőrizzük, hogy elég nagy-e aztán elvégezzük a buffer feltöltését a processBlock()-ban.

3.2. Anti-Imaging Szűrés

A nullákkal feltöltött jel "tüskés", és tele van nemkívánt magas frekvenciás komponensekkel. Az upsampleFIR szűrő simítja a jelet, kitöltve a nullákat a szomszédos minták átlagolásával (interpoláció).

Kódrészlet:

```

// 2. Anti-Imaging szűrő (Upsampling után kötelező, különben tükörfrekvenciák lesznek)
{
    juce::dsp::AudioBlock<float> block(upsampledBuffer);
    // Csak a releváns mintákat dolgozzuk fel
    juce::dsp::AudioBlock<float> subBlock = block.getSubBlock(0, numInputSamples * L);
    juce::dsp::ProcessContextReplacing<float> ctx(subBlock);
    upsampleFIR.process(ctx);
}

```

Itt szűröm meg a interpolált mintákat

3.3. Nemlineáris Torzítás (Distortion)

Ez az effekt "szíve". A tiszta, felskálázott jelen végrehajtunk egy hard clipping műveletet.

- A bemenetet megszorozzuk a csúszkán változtatható GAIN paraméterrel.
- Az eredményt levágjuk (clamp) -0.6 és +0.6 közé. Ez a vágás hozza létre a jellegzetes, agresszív gitártorzítást. Mivel 4x-es mintavételi frekvencián vagyunk, a keletkező harmonikusoknak van "helyük" létezni anélkül, hogy aliasing lépne fel.

Kódrészlet:

```
float NonlinearDistortionAudioProcessor::applyDistortion(float input, float gain)
{
    return std::clamp(gain * input, -0.6f, 0.6f);
}
```

Ezt a függvényt hívom meg minden egyes mintára.

```
// 3. Torzítás
for (int ch = 0; ch < upsampledBuffer.getNumChannels(); ++ch)
{
    auto* data = upsampledBuffer.getWritePointer(ch);
    // Fontos: csak addig iteráljunk, amennyi minta ténylegesen van
    for (int n = 0; n < numInputSamples * L; ++n)
        data[n] = applyDistortion(data[n], gain);
}
```

Itt megyek végig minden mintát és alkalmazom rá a hard-clipping-et.

3.4. Anti-Aliasing Szűrés

Mielőtt visszatérnénk az eredeti mintavételi frekvenciára, el kell távolítanunk minden olyan frekvenciát, ami az eredeti $f_s/2$ felett van. Ha ezt nem tennénk meg, a következő lépésben (decimálás) ezek a magas hangok "visszahajlanának" mély hangokként, zajt okozva.

Kódrészlet:

```
// 4. Anti-Aliasing szűrő (Decimálás előtt)
{
    juce::dsp::AudioBlock<float> block(upsampledBuffer);
    juce::dsp::AudioBlock<float> subBlock = block.getSubBlock(0, numInputSamples * L);
    juce::dsp::ProcessContextReplacing<float> ctx(subBlock);
    downsampleFIR.process(ctx);
}
```

1 A decimálás és interpolálás előtt is processContext-be kell csomagolni a buffer-t, mert az FIR szűrő osztály process függvény-t nem lehet egyenesen bufferre alkalmazni

3.5. Decimálás (Downsampling)

Végül eldobáljuk a felesleges mintákat, csak minden L-ediket tartjuk meg. Mivel az előző lépésben kiszűrtük a magas frekvenciákat, ez a lépés már biztonságos.

Kódrészlet:

```
void NonlinearDistortionAudioProcessor::filldownsampleBuffer(
    const juce::AudioBuffer<float>& input,
    juce::AudioBuffer<float>& downsampled,
    int L)
{
    int numOutputSamples = downsampled.getNumSamples();
    int numChannels = input.getNumChannels();

    for (int ch = 0; ch < numChannels; ++ch)
    {
        auto* in = input.getReadPointer(ch);
        auto* out = downsampled.getWritePointer(ch);

        for (int n = 0; n < numOutputSamples; ++n)
            out[n] = in[n * L];
    }
}
```

2A függvényimplementáció amivel feltöltöm a decimáláshoz használatos buffer-t

```
// 5. Decimálás
filldownsampleBuffer(upsampledBuffer, downsampledBuffer, L);
```

3 Decimálás függvényhívása.

3.6 Az eredeti bufferbe visszírjuk a torzított mintákat:

Miután megtörtént a teljes procedúra, visszaírjuk a processBlock paramétereként megadott kimeneti bufferbe a mintákat, hogy hallhatóvá váljanak.

Kódrészlet:

```
void NonlinearDistortionAudioProcessor::copyToOutput(
    const juce::AudioBuffer<float>& src,
    juce::AudioBuffer<float>& dst)
{
    int numChannels = dst.getNumChannels();
    int numSamples = dst.getNumSamples();

    for (int ch = 0; ch < numChannels; ++ch)
    {
        std::memcpy(dst.getWritePointer(ch),
                    src.getReadPointer(ch),
                    numSamples * sizeof(float));
    }
}
```

4 Eredeti bufferbe írás függvény implementációja

```
//Eredeti bufferbe való viisszaírás  
copyToOutput(downsampledBuffer, buffer);
```

5 Függvényhívás

4. Paraméterkezelés (AudioProcessorValueTreeState)

A modern JUCE szabványoknak megfelelően a paramétereket (jelen esetben a GAIN-t) az AudioProcessorValueTreeState (APVTS) kezeli. Ez biztosítja a szálbiztos kommunikációt a GUI és az audio szál között.

Kódrészlet:

```
juce::AudioProcessorValueTreeState::ParameterLayout NonlinearDistortionAudioProcessor::createParameters()  
{  
    std::vector<std::unique_ptr<juce::RangedAudioParameter>> params;  
  
    params.push_back(std::make_unique<juce::AudioParameterFloat>(  
        "GAIN",    // azonos  
        "Gain",    // felirat a GUI-n  
        1.0f,      // minimum  
        50.0f,     // maximum  
        10.0f));   // default érték  
  
    return { params.begin(), params.end() };  
}
```

A processBlock-ban a *apvts.getRawParameterValue("GAIN") hívással kérjük le az aktuális értéket atomi művelettel, így nem akasztjuk meg az audio folyamatot.

5. Használt eszközök, irodalmak:

- Jelek és Rendszerek tankönyv: https://vik.wiki/images/a/ad/Jelek_konyv_1.pdf
- Hibakeresés, JUCE szintaxissal kapcsolatos segítség: <https://gemini.google.com/app>
- Richard G. Lyons - Understanding Digital Signal Processing:
<https://www.iro.umontreal.ca/~mignotte/IFT3205/Documents/UnderstandingDigitalSignalProcessing.pdf>